# EN3160 Assignment 2 on Fitting and Alignment

**Name : Vikkramanayaka A.G.P.S** **Index : 200683X**

**GitHub Link :** [pasindu201/image-Processing-and-Machine-Vision: Image Processing and Machine Vision is a semester 5 course module and this repository contains all the related codes and projects. (github.com)]

## Question 1

```
# define log kernel
def log_kernel(sigma, size):
    if size % 2 == 0:
        size += 1
    sigma2
    idx_ran (variable) y_idx: NDArray[Any] ., (size - 1) / 2., size)
    x_idx, y_idx = np.meshgrid(idx_range, idx_range)
    tmp_cal = -(np.square(x_idx) + np.square(y_idx)) / (2. * sigma2)
    kernel = np.exp(tmp_cal)
    kernel[kernel < np.finfo(float).eps * np.amax(kernel)] = 0
    k_sum = np.sum(kernel)
    if k_sum != 0:
        kernel /= np.sum(kernel)
    tmp_kernel = np.multiply(kernel, np.square(x_idx) + np.square(y_idx) - 2 * sigma2) / (sigma2 ** 2)
    kernel = tmp_kernel - np.sum(tmp_kernel) / (size ** 2)
    return kernel
```

The algorithm starts by defining a LoG kernel. And checks range of σ values starting from $\sqrt{2}$ and powers of $\sqrt{2}$ up to $8\sqrt{2}$.

Range of σ values used: $\sqrt{2}$ , 2, $2\sqrt{2}$, 4 , $4\sqrt{2}$, 8 , $8\sqrt{2}$

In my algorithm input image is converted to grayscale and normalized. Then computing the convolution of the image with the LoG kernel at multiple scales. For each scale, the LoG kernel is applied to the image to enhance regions with varying intensity, effectively highlighting potential blob centers. The algorithm extracts the maximum response from the filtered images at each scale. Then local maxima are identified as potential blob centers. The algorithm records the coordinates (X, Y) and radii of the detected blobs for all scales.

```
# blob detection
def detect_blobs(img, sigma_scales, threshold):
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    gray = np.float32(gray)
    cv.normalize(gray, gray, 1, 0, cv.NORM_MINMAX)

    sigma0 = np.sqrt(2)
    k = np.sqrt(2)
    num_scales = sigma_scales
    sigmas = sigma0 * np.power(k, np.arange(num_scales))
    sigmas = [num_scales]*num_scales


    # apply LoG kernel filtering with scaled kernel size and sigma
    img_stack = None
    for i in range(num_scales):
        size = int(2 * np.ceil(4 * sigmas[i]) + 1)
        # with Laplacian response normalization
        kernel = log_kernel(sigmas[i], size) * np.power(sigmas[i], 2)
        filtered = cv.filter2D(gray, cv.CV_32F, kernel)
        filtered = pow(filtered, 2)
        if i == 0:
            img_stack = filtered
        else:
            img_stack = np.dstack((img_stack, filtered))
```

```
# Maximum response extraction
scale_space = None
for i in range(num_scales):
    filtered = cv.dilate(img_stack[:, :, i], np.ones((3, 3)), cv.CV_32F, (-1, -1), 1, cv.BORDER_CONSTANT)
    if i == 0:
        scale_space = filtered
    else:
        scale_space = np.dstack((scale_space, filtered))
max_stack = np.amax(scale_space, axis=2)
max_stack = np.repeat(max_stack[:, :, np.newaxis], num_scales, axis=2)
max_stack = np.multiply((max_stack == scale_space), scale_space)

radius_vec = None
x_vec = None
y_vec = None
for i in range(num_scales):
    radius = np.sqrt(2) * sigmas[i]
    threshold = threshold
    # filter out redundant response
    valid = (max_stack[:, :, i] == img_stack[:, :, i]) * img_stack[:, :, i]
    valid[valid <= threshold] = 0
    (x, y) = np.nonzero(valid)
    if i == 1:
        x_vec = x
        y_vec = y
        radius_vec = np.repeat(radius, np.size(x))
    else:
```

<u>Parameters of the largest circles</u>

Radius - 11.313708498984761

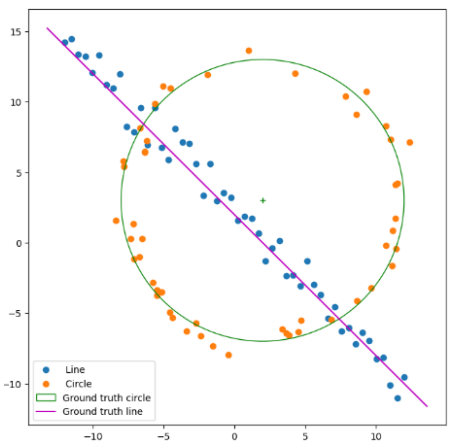
Corresponding sigma - 8.0

```python
            x_vec = np.concatenate((x_vec, x), axis=None)
            y_vec = np.concatenate((y_vec, y), axis=None)
            tmp_vec = np.repeat(radius, np.size(x))
            radius_vec = np.concatenate((radius_vec, tmp_vec), axis=None)

    return x_vec, y_vec, radius_vec

img = cv.imread('the_berry_farms_sunflower_field.jpeg', cv.IMREAD_REDUCED_COLOR_4)
x_all = []
y_all = []
radii_all = []
for sigma_scale in range(2, 10):
    x_coords, y_coords, radii = detect_blobs(img, sigma_scale, 0.1)
    x_all.append(x_coords)
    y_all.append(y_coords)
    radii_all.append(radii)

x_all = np.concatenate(x_all, axis=None)
y_all = np.concatenate(y_all, axis=None)
radii_all = np.concatenate(radii_all, axis=None)

output_img = img.copy()
for i in range(int(len(x_all))):
    cv.circle(output_img, (y_all[i], x_all[i]), int(radii_all[i]), (0, 0, 255), 1)
```

Largest blob detection code

```python
largest_blob_idx = np.argmax(radii_all)

# Get the coordinates and radius of the largest blob
largest_blob_x = x_all[largest_blob_idx]
largest_blob_y = y_all[largest_blob_idx]
largest_blob_radius = radii_all[largest_blob_idx]

# Create a copy of the input image
output_img_with_largest_blob = img.copy()

# Draw the largest blob on the image
cv.circle(output_img_with_largest_blob, (largest_blob_y, largest_blob_x), int(largest_blob_radius), (0, 0, 255), 2)
```

**Question 2**




In this assignment question, we begin with a dataset containing both noisy data points belonging to a line and noisy data points belonging to a circle. The objective is to estimate the line, given that we initially have no information about which points belong to the line and which are outliers.

To achieve this, I have used the Random Sample Consensus (RANSAC) algorithm. RANSAC operates by iteratively selecting pairs of data points at random and fitting a line that passes through those two points. Subsequently, it measures the distance from every data point to the fitted line. If the distance is below a specified threshold the point is considered a consensus point, indicating that it likely belongs to the line model. This process is repeated for multiple random pairs.

Once enough consensus points are accumulated, we can conclude that this set of consensus points forms the estimated line. This method helps us robustly identify which data points are part of the line, even in the presence of outliers.

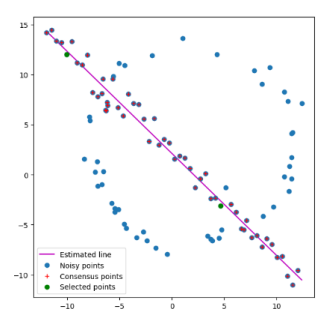
By applying RANSAC in this manner, we can extract the inliers (points belonging to the line) from the dataset, facilitating accurate line estimation.

```python
num_iterations = 1000
consensus_threshold = 1.6 # Maximum distance to say that a point is a inlier
min_samples = 2 # Minimum number of points to describe a model
# Define a function to calculate the distance from a point to a line
def distance_to_line(point, line_parameters):
    a, b, d = line_parameters
    x, y = point
    return np.abs(a * x + b * y -d) / np.sqrt(a**2 + b**2)
#Define a line goes through two points
def line(x,y):
    m = (y[1] - y[0])/(x[1] - x[0])
    c = y[0] - m*x[0]
    a = -m
    b = 1
    d = c
    return  a , b , d
points = all_points
for _ in range(num_iterations):
    # Randomly select two points
    selected_indices_L = np.random.choice(len(points), size=min_samples, replace=False)
    selected_points_L = points[selected_indices_L]
    # Fit a line to the selected points and get the line parameters
    a, b, d = line(selected_points_L[:,0], selected_points_L[:,1])
    # (a**2 + b**2) = 1 =>
    norm = np.sqrt(a**2 + b**2)
    a /= norm
    b /= norm
    d /= norm
    consensus_set_L = []
    # Find the consensus set by checking each point against the line
    for i, point in enumerate(points):
        if distance_to_line(point, [a, b, d]) < consensus_threshold:
            consensus_set_L.append(i)
    if len(consensus_set_L) >45:
        break
```

Then using the selected consensus points we can estimate the best line. For that we use the equation,

$$(X^{\mathrm{T}} . X) -1 . (X^{\mathrm{T}} . Y)$$

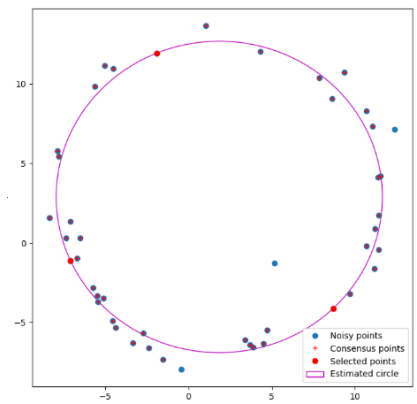
```python
A = np.hstack((xc_L.reshape(len(xc_L),1), np.ones((len(xc_L),1))))
b_ = yc_L.reshape(len(yc_L),1)
xstar = np.linalg.inv(A.T.dot(A)).dot(A.T).dot(b_)
mstar = xstar[0]
cstar = xstar[1]
```




- Estimated line: y $= -1.\mathbf{020x} + \mathbf{2.124}$
- Ground truth line: y = -x + 2

Following the same method, we can determine the circle goes through the remaining points.

```python
np.random.seed (0)
num_iterations = 1000
consensus_threshold = 1.1 # Maximum distance to say that a point is an inlier
min_samples = 3 # Minimum number of points to describe a model
# Define a function to calculate the distance from a point to a circle
def distance_to_circle(point, circle_parameters):
    x0, y0, r = circle_parameters
    x, y = point
    return np.abs(np.sqrt((x - x0)**2 + (y - y0)**2) - r)
# Define a function to fit a circle through three points
def circle(x, y):
    x1, x2, x3 = x
    y1, y2, y3 = y
    A = np.array([[2*(x2-x1), 2*(y2-y1)], [2*(x3-x2), 2*(y3-y2)]])
    b = np.array([x2**2 + y2**2 - x1**2 - y1**2, x3**2 + y3**2 - x2**2 - y2**2])
    x0, y0 = np.linalg.inv(A).dot(b)
    r = np.sqrt((x1-x0)**2 + (y1-y0)**2)
    return x0, y0, r
new_points = remaining_points
estimated_circle = None
for _ in range(num_iterations):
    # Randomly select three points from the dataset
    selected_indices_C = np.random.choice(len(new_points), size=min_samples, replace=False)
    seletecd_points_C = new_points[selected_indi (variable) seletecd_points_C: ndarray[Any, dtype[floating[Any]]]
    # Fit a circle to the selected points
    x0, y0, r = circle(seletecd_points_C[:, 0], seletecd_points_C[:, 1])
    consensus_set_C = []
    # Find the consensus set by checking each point against the circle
    for i, point in enumerate(new_points):
        if distance_to_circle(point, [x0, y0, r]) < consensus_threshold:
            consensus_set_C.append(i)
    if len(consensus_set_C) > 42:
        estimated_circle = [x0, y0, r]
        break
```

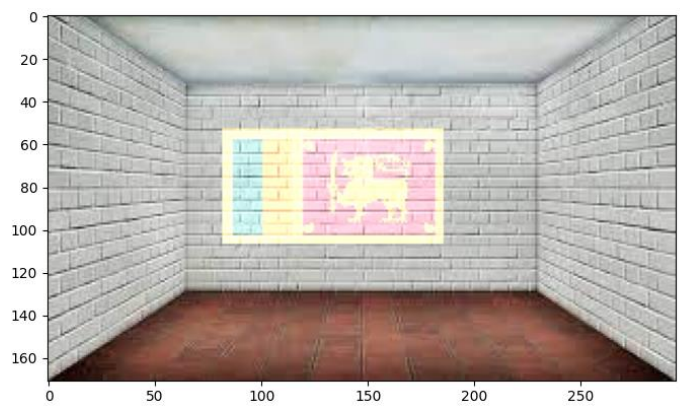



## Question 3

In this exercise we need to superimpose a flag image into architectural image. The first step is to choose four points on the architectural image that correspond to the four corner points of the flag image.

```python
architectural_image = cv.imread('images/005.jpg')
flag = cv.imread('Flag_of_the_United_Kingdom_(1-2).svg.png')

architectural_image = architectural_image.astype(np.float32) / np.iinfo(architectural_image.dtype).max
flag = flag.astype(np.float32) / np.iinfo(flag.dtype).max

print("Image shape:", architectural_image.shape)
print("Flag shape:", flag.shape)

cv2.imshow('Image', architectural_image)
cv2.setMouseCallback('Image', mouse_click)

# Wait for the user to select four points
while len(selected_points) < 4:
    cv2.waitKey(1)

cv2.destroyAllWindows()

print("Selected Points:", selected_points)

dst_points = np.array(selected_points).astype(np.float32)
h2, w2 = flag.shape[:2]
src_points = np.float32([[0, 0], [w2, 0], [0, h2], [w2, h2]])

H = cv2.getPerspectiveTransform( src_points,dst_points) #homograph matrix
print('Homograph matrix\n',H)

flag_warped = cv2.warpPerspective(flag, H, (architectural_image.shape[1], architectural_image.shape[0]))
superimposed_image = cv.addWeighted(architectural_image, 1, flag_warped, 0.5, 0)
```

In this case, I have superimposed the Sri Lankan flag onto a wall.



```python
def findFeatures(img):
    sift = cv.SIFT_create()
    key, des = sift.detectAndCompute(img, None)
    return key, des

def findMatches(des1, des5):
    bf = cv.BFMatcher()
    matches = bf.knnMatch(des1, des5, k=2)
    return matches


def calHomography(sample):
    A = []
    for i in range(len(sample)):
        x1, y1, x2, y2 = sample[i, 0], sa (variable) y2: Any , 2], sample[i, 3]
        A.append([-x1, -y1, -1, 0, 0, 0,
        A.append([0, 0, 0, -x1, -y1, -1, y2*x1, y2*y1, y2])

    A = np.matrix(A)
    U, S, V = np.linalg.svd(A)
    H = np.reshape(V[-1], (3, 3))
    H = (1/H.item(8))*H
    return H
```

## Question 4

Here I have used a SIFT detector to extract key features from both images and create descriptors for those

features. Then I used FlannBasedMatcher to match features and filtered only good matches.

```
img1_gray = cv.cvtColor(img1, cv.COLOR_BGR2GRAY)
img5_gray = cv.cvtColor(img5, cv.COLOR_BGR2GRAY)
img1 = cv.cvtColor(img1, cv.COLOR_BGR2RGB)
img5 = cv.cvtColor(img5, cv.COLOR_BGR2RGB)

sift = cv.SIFT_create()
bf = cv.BFMatcher()

key1, des1 = sift.detectAndCompute(img1_gray, None)
key5, des5 = sift.detectAndCompute(img5_gray, None)

matches = bf.knnMatch(des1, des5, k=2)
print(matches[:5])
matches = sorted(matches, key=lambda x:x[0].distance)

out_img = cv.drawMatchesKnn(img1, key1, img5, key5, matches[:200], img5, flags=2)
```

```
def calDistance(H, correspondence):
    p1 = np.transpose(np.matrix([correspondence[0].item(0), correspondence[0].item(1), 1]))
    estimatep2 = np.dot(H, p1)
    estimatep2 = (1/estimatep2.item(2))*estimatep2

    p2 = np.transpose(np.matrix([correspondence[0].item(2), correspondence[0].item(3), 1]))
    error = p2 - estimatep2
    return np.linalg.norm(error)


def RANSAC(corres,threshold):
    max_inliers = []
    homography = []

    #(int(np.log(1 - 0.95)/np.log(1 - (1 - 0.5)**4)))

    for i in range(100):
        corr1 = corres[np.random.randint(0, len(corres))]
        corr2 = corres[np.random.randint(0, len(corres))]
        sample = np.vstack((corr1, corr2))
        corr3 = corres[np.random.randint(0, len(corres))]
        sample = np.vstack((sample, corr3))
        corr4 = corres[np.random.randint(0, len(corres))]
        sample = np.vstack((sample, corr4))
```

```
        h = calHomography(sample)
        inliers = np.zeros((1,4))

        for j in range(len(corres)):
            distance = calDistance(h, corres[j])
            if distance < 5:
                inliers = np.vstack((inliers,corres[j]))

        inliers = np.delete(inliers,0,0)

        if len(inliers) > threshold:
            max_inliers = inliers
            homography = calHomography(max_inliers)

    return homography,max_inliers
```

Estimated Homography Matrix:

[[ 6.22687296e-01  6.64753688e-02  2.19896547e+02]

 [ 2.16759562e-01  1.14463617e+00 -1.71586629e+01]

 [ 4.89265935e-04 -4.18045527e-05  1.00000000e+00]]

Given Homography Matrix:

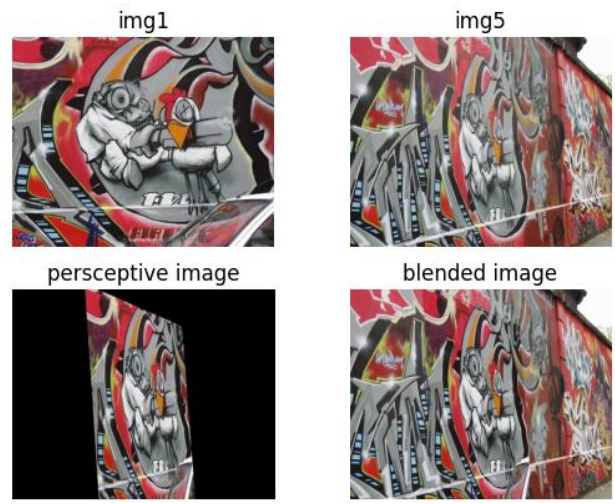
6.2544644e-01  5.7759174e-02  2.2201217e+02

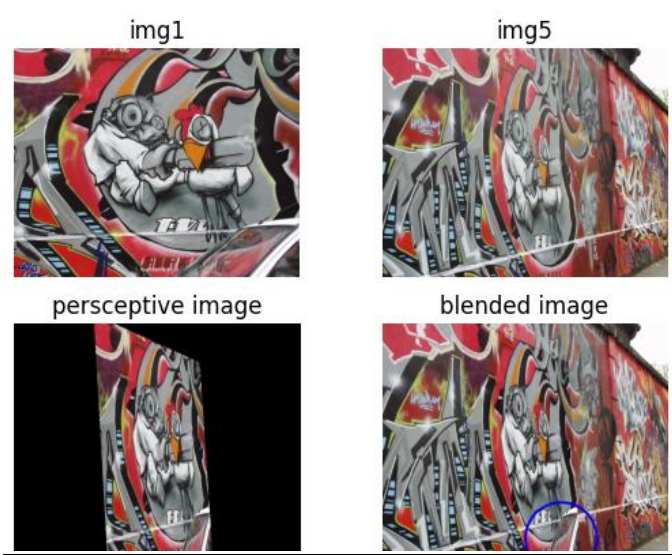
2.2240536e-01  1.1652147e+00  -2.5605611e+01

4.9212545e-04  -3.6542424e-05  1.0000000e+00

## Sketching images using calculated Harmograpy

```
threshold, mask = cv.threshold(cv.cvtColor(image_perspective, cv.COLOR_BGR2GRAY), 1, 255, cv.THRESH_BINARY)
mask_inv = cv.bitwise_not(mask)
img5_bg = cv.bitwise_and(img5, img5, mask=mask_inv)
dst = cv.addWeighted(img5_bg, 1, image_perspective, 1, 0)
```




## Sketching images using original Harmograpy

```
#coverting h_text to homography matrix
original_H = []
for line in h_text:
    line = line.strip()
    line = line.split()
    line = [float(i) for i in line]
    original_H.append(line)


original_H = np.matrix(original_H)

image_perspective = cv.warpPerspective(img1, original_H, (img5.shape[1], img5.shape[0]))
plt.subplot(2,2,3)
plt.imshow(cv.cvtColor(image_perspective, cv.COLOR_BGR2RGB))
plt.title('persceptive image')
plt.axis('off')

threshold, mask = cv.threshold(cv.cvtColor(image_perspective, cv.COLOR_BGR2GRAY), 1, 255, cv.THRESH_BINARY)
mask_inv = cv.bitwise_not(mask)
img5_bg = cv.bitwise_and(img5, img5, mask=mask_inv)
dst = cv.addWeighted(img5_bg, 1, image_perspective, 1, 0)
output_image = cv.cvtColor(dst, cv.COLOR_BGR2RGB)
difference = cv.circle(output_image, (500, 600), 100, (0, 0, 255), 5)
```